- Form fields are rendered by widgets, with each field having a default widget that renders the corresponding HTML.
- The choice of a widget for a field can be overridden, like in the case of a `forms.CharField` using a `forms.Textarea` widget to be rendered as a `<textarea>` instead of an `<input>`.
- The `request.method` attribute contains the HTTP method.
- A view that handles forms typically has two modes: `GET` for displaying the form and `POST` for handling the submission.
- A `Form` object can be filled with the user's submitted data by passing in the `request.POST` value.
- The `django.core.mail` module contains the `send_mail()` function, which can be used to send email using the currently configured email backend.
- When processing a form, the `.is_valid()` method must be called. This method returns `True` if the data is compliant with the form's definition.
- When `.is_valid()` returns `True`, a dictionary called `.cleaned_data` is defined on the form containing the user's submitted input. The keys in the dictionary correspond to the names of the fields.
- A cross-site request forgery is an attack that tricks the user into doing something on your site they didn't intend. The `{% csrf_token %}` tag adds a special key to every form to help prevent this kind of attack.
- The `.clean()` method on a `Model` class can be used to do validation. It is typically used when there are interdependencies between the fields and field-level validation arguments are insufficient.
- The `.clean()` method is not called automatically when an ORM class is saved.
- The undocumented `Truncator` class is a utility that returns a shortened version of a string, truncating on word boundaries.
- The `ModelForm` class provides a shortcut for creating forms that correspond to Model definitions.
- A `ModelForm` uses an inner `Meta` class to indicate the configuration of the form, including which model is related and which fields on the model participate in the form.
- The `.save()` method on a `ModelForm` saves the corresponding model object. This method can be called with `commit=False` to return an unsaved copy of the model instance if you need to perform actions before invoking save.
- The `instance` argument to a `ModelForm` allows you to prepopulate the field contents based on a model object.
- You can register multiple routes against the same view. This is handy if your view uses default arguments. A common pattern is to have a view for editing be able to create new objects as well using an ID of zero to indicate creation.
- Web pages reference files that aren't produced by views, such as images, CSS, and JavaScript. These are known as static files.